Introduction to NLP, Exercise 3

Sabine Weber

Publication: Nov 3, 2026
Submission: Nov 10, 2026
Discussion: Nov 12, 2026

- **Deadline:** The submission needs to be done via VC until the end of the day mentioned on the PDF. A late submission is not possible.
- **Groups:** You are allowed to work in groups. A group can consist of maximally three members. The members can change in each submission. The names that you put on the PDF are what we consider as the group.
- **Grading:** We plan to offer 11 exercise sheets. For each sheet, you can achieve 20 points. From the 220 points altogether, we will consider 176 points to be a perfect participation (if, for some reason, we offer fewer assignments, this number of required points will be reduced proportionally). The assignment submissions will contribute up to 20 % of the grade as bonus points.
- **Submission:** Only submit one PDF file. Program code should be part of that PDF file. Put all names and email addresses of your group members on the first page of this PDF file. Please do submit the PDF only once (not multiple times by multiple group members).
- **Sources:** We recommend to not use any "AI" or large language model for creating the solution to any subtask/task as a whole. You can use other sources (documentation, papers, stackexchange, ...). If you decide to use an "AI" or LLM (like ChatGPT, Hawki) for a part of your solution, provide the information which system you used, provide the complete prompt and the complete answer by the automatic system (in German or English) in an appendix of your submission. I need to be able to understand what your own contribution on top of the automatic result was. You only receive points for your own contribution on top of what you got from the source you provide. If you use an "AI" and do not provide the required reference or do not make that use transparent, we will consider this plagiarism.
- **Plagiarism:** When two groups submit solutions that are copied from each other or from a third source that is not made transparent in the submission, all these groups do not receive any points. The same holds if one group copies from a source that is no provided in the submission. The whole submission will be considered invalid, not only the task in which we detected the plagiarism. No points will be given for the whole submission. These sanctions do not exclude further measures.

Do you have questions? Send me an email (sabine.weber@uni-bamberg.de) or use the forum.

Task 1 - Pen & Paper (10 points)

A: Find one tagging error in each of the following sentences, which are tagged with the Penn Treebank tagset:

1. I/PRP need/VBP a/DT flight/NN from/IN Atlanta/NN
2. Does/VBZ this/DT flight/NN serve/VB dinner/NNS
3. I/PRP have/VB a/DT friend/NN living/VBG in/IN Denver/NNP
4. Can/VBP you/PRP list/VB the/DT nonstop/JJ afternoon/NN flights/NNS

B: Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation.

1. It is a nice night.
2. This crap game is over a garage in Fifty-second Street.
3. Nobody ever takes the newspapers she sells.
4. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.
5. I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.

C: Create constituency parse trees for the sentences from part B. You can use the simplified grammar in the lecture slides.
D: Consider the sentence "I saw the man with a telescope."

1. Assign Penn Treebank POS tags to all words in the sentence.
2. Create two different constituency parse trees for the sentence, corresponding to the two possible interpretations.
3. Come up with 5 additional sentences that have the same POS tags but several possible constituency parse trees.

Task 2 - Programming (10 points)

A: Implement the "most likely tag" baseline for the task of POS tagging. Find a POS-tagged corpus, and use it to compute for each word w the tag t that maximizes $p(t|w)$.
B: Build a HMM tagger. You can use the same corpus as in part A. First split the corpus into a training set and test set. From the training set, count how often each POS tag occurs, how often one POS tag follows another and how often each word occurs with each POS tag. Compute the transition and observation probabilities of the HMM tagger from this. Then implement the Viterbi algorithm so you can decode a test sentence. Create a Viterbi table that stores, for each position in the sentence and each possible tag, the probability of the most probable path ending in that tag. Initialize the table for the first word using the initial tag

probabilities and the observation probability of that word. Process the remaining words from left to right. For each word, update the Viterbi table by considering all possible tags and the probabilities of the most probable paths leading to them. Store both the best score and the preceding tag that achieved it (the backpointer). After filling the table, backtrack from the last word to find the most probable sequence of tags.

Now run your algorithm on the test set. Report its error rate and compare its performance to the most frequent tag baseline.

C: Find or create 10 challenging test sentences for your HMM tagger. Justify why each of them might be difficult to a HMM tagger. Then, test them with the tagger and see if your implementation handles them correctly, and try to explain any errors that occur.